propPitch: how it works

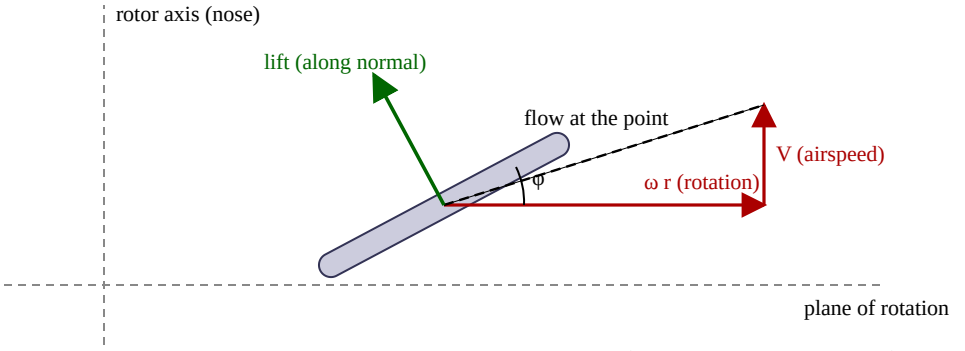
Holds a propeller plane's collective blade pitch at the angle that gives the most forward thrust, computed every tick from the game's own blade aerodynamics. It flies only the propellers.

Phases



START finds every robotic rotor and the control-surface blades below it, disables the craft's KAL controllers, runs every rotor at `ROTOR_RPM` / `ROTOR_TORQUE_PCT` with its motor engaged, and reads each blade's position and axes once (vessel frame, relative to its rotor). Blade part names are looked up in `bladeTable.py`; an unknown blade stops START with its name. **TRACK** commands the optimum every tick and re-asserts the KALs' and rotors' settings every `REASSERT_S`. **TERMINATE** re-enables the KALs and restores each rotor's rpm and torque limits.

The blade law (KSP's, evaluated off-line)



One blade, seen along its span. Thrust is lift's and drag's components along the nose.

KSP treats each blade as one lifting surface evaluated at one point: the part's `velocityOffset` along its span (−5.8 m on the R-25, so about 6.0 m from the hub). With u the direction of that point's velocity through the air, n the blade's normal turned by s·Deploy Angle about its span, $c = u \cdot n$, $q = \frac{1}{2}\rho W^2$ and $M = W/a$ of that point:

$$\text{lift} = -n \cdot \text{sign}(c) \cdot \text{Cl}(|c|) \cdot \text{Ml}(M) \cdot k \cdot q \cdot 0.036 \quad \text{(applied at the part's CoL offset)}$$
$$\text{drag} = -u \cdot \text{Cd}(|c|) \cdot \text{Md}(M) \cdot k \cdot q \cdot 0.015 \quad \text{(applied at the part's origin)}$$

k = deflectionLiftCoeff × ctrlSurfaceArea; the four curves are the part cfg's (FloatCurves, cubic Hermite); 0.036 and 0.015 are Physics.cfg's lift and lift-drag multipliers. Lift is along the normal, not perpendicular to the flow. The point's velocity is the vessel's air velocity (read in the vessel's

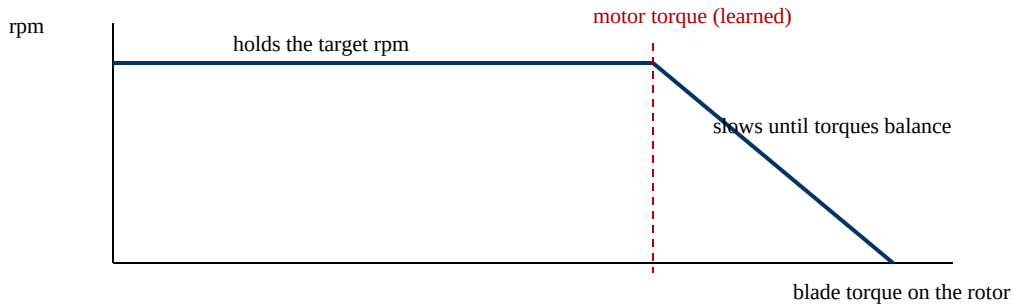
axes through a hybrid frame) plus ω×r. The four blades of a rotor sit 90° apart, so their geometry at any one moment is an average around the disc.

Deflection sense *s*: kRPC does not expose the blade's CW/CCW variant or which way the rotor turns. At START, for each blade, *s* is the sign that gives forward thrust at rest at 60/75/85°. Thrust depends only on the product of *s* and the spin direction.

The search

Every tick: total thrust along the nose at Deploy Angle 0…90 on a 1° grid (`SEARCH_STEP_DEG`), then golden section around the best cell to `SEARCH_TOL_DEG`. The angle found is commanded on every blade; KSP applies Deploy Angle without an actuator delay.

Load and rpm



Each angle is scored at the rpm its rotor would settle at.

KSP's rotor is a velocity drive (damper 1000) whose torque is capped at `maxTorque` × torque limit (70 kN·m on the EM-32S at 100%), and zero without electric charge. Its charge draw, 0.06 × 70 × limit × rpm/460 / 0.75 EC/s, does not depend on load. For each angle, the model computes the blades' torque at the target rpm. If that exceeds the rotor's motor torque, it bisects for the rpm where the two balance and scores the thrust there.

The motor torque starts at the game's cap and is learned from the measured rpm (`Rotor.observe`). When a rotor runs more than 2% below its target and its rpm readout has stayed flat for 0.6 s, the model's blade torque at that rpm becomes the motor torque (smoothed over 0.5 s). The 0.6 s is there because the game refreshes the readout every 0.3 s, so a rotor spinning up reads flat between refreshes. While the rotor holds its target, the estimate is at least the current load and recovers toward the cap over 5 s. A motor without power learns ~0, and the search then picks the angle that loads that rotor least.

The log

field	what
`pitch=` / `opt=`	angle flown / the model's optimum
`J=`	measured forward aero force per unit mass (thrust less airframe drag), m/s²
`model=`	the model's blade thrust per unit mass at the angle flown
`rpm=a/b`	measured mean rpm / the rpm the model expects the slowest rotor to settle at

`torque=`	the worst rotor's blade torque over the game's cap
`motor=`	learned motor torque per rotor, kN·m
`va=`	velocity through the air in the vessel's axes (x right, y nose, z down)

J is read before the tick's command, so it answers the previous tick's angle. `tools/bladeprobe.py` replaces the command with a sweep, and `tools/probesum.py` compares J with the model at each angle held.